Full Stack Development with MERN

Project Documentation format

1. Introduction

- **Project Title: Automating Employee Data**
- **Team Members:**
  - Ramviswa,Praveen Kumar – **Frontend Developer**
  - Ajmal,Daniel Prem Kumar – **Backend Developer**
  - Muthu – **Database & Testing**

---

2. Project Overview

Purpose:

The purpose of this project is to automate the management of employee data, including data entry, storage, processing, and visualization using a web-based system.

Features:

- **Employee Registration & Login**
- **Add / Edit / Delete Employee Data**
- **Dashboard with Data Visualization**
- **Filtering and Search Options**
- **Secure Authentication System**

---

3. Architecture

Frontend (React):

- **Built using React.js**
- **Uses components for UI (Login, Dashboard, Forms)**
- **Axios used for API communication**

Backend (Node.js + Express):

- **REST API built using Express.js**
- **Handles requests, authentication, and business logic**

Database (MongoDB):

- **Stores employee data (Name, ID, Salary, Department, etc.)**
- **Uses collections and documents**
- **Mongoose used for schema design**

---

4. Setup Instructions

Prerequisites:

- **Node.js**
- **MongoDB**
- **Git**

Installation:

1. **Clone the repository**

```
git clone https://github.com/your-repo/employee-data.git
```

2. **Navigate to project folder**

```
cd employee-data
```

3. **Install dependencies**

```
npm install
cd client && npm install
```

4. **Setup environment variables (.env file)**
    - **MongoDB URI**
    - **JWT Secret**

---

5. Folder Structure

Client (React):

- **src/components → UI Components**
- **src/pages → Pages (Login, Dashboard)**
- **src/services → API calls**

Server (Node.js):

- **routes → API routes**
- **controllers → Business logic**
- **models → Database schema**
- **config → Database connection**

---

6. Running the Application

Frontend:

```
cd client
npm start
```

Backend:

```
cd server
npm start
```

---

7. API Documentation

Method	Endpoint	Description
POST	/api/register	Register user
POST	/api/login	Login user
GET	/api/employees	Get all employees

Method	Endpoint	Description
POST	/api/employees	Add employee
PUT	/api/employees/:id	Update employee
DELETE	/api/employees/:id	Delete employee

---

8. Authentication

- Uses JWT (JSON Web Token)
- Token generated during login
- Protected routes require token
- Passwords stored securely (hashed)

---

9. User Interface

- Simple and responsive UI
- Pages include:
  - Login Page
  - Registration Page
  - Dashboard
  - Employee Management Page

(Add screenshots here for submission)

---

10. Testing

- **Manual testing performed**
- **Functional testing for all modules**
- **Performance testing for dashboard loading**
- **Tools: Browser DevTools / Postman**

---

11. Screenshots / Demo

- **Dashboard View**
- **Employee List**
- **Charts & Reports**

(Attach screenshots or demo link)

---

12. Known Issues

- **Minor delay in dashboard loading**
- **Limited scalability for large datasets**

---

13. Future Enhancements

- **Add role-based access (Admin/User)**
- **Improve UI design**
- **Add real-time data updates**
- **Integrate advanced analytics**